BrewFlow Final Project

Teil 1 OLTP Datenbank

Projektdokumentation zur Einrichtung und Validierung des Schemas brewflow_oltp

Ziel

Ziel von Teil 1 ist der Aufbau einer relationalen OLTP-Datenbank für BrewFlow. Kunden, Filialen, Produkte, Bestellungen und Bestellpositionen werden in fünf verknüpften Tabellen gespeichert. Die Datenbank bildet einzelne Verkaufsvorgänge ab und stellt die Grundlage für weitere Projektschritte bereit.

Teil 1 ist abgeschlossen: Alle fünf Tabellen sind angelegt und mit Beispieldaten befüllt. Die Zeilenanzahlen entsprechen den Vorgaben. Bei allen 22 Bestellungen stimmt die gespeicherte Bestellsumme mit der Summe der zugehörigen Positionen überein.

Teil A Schema erstellen

Aufgabenstellung Einen eigenen Namensraum für die BrewFlow-OLTP-Tabellen anlegen.

Schritt Schema erstellen und in der aktuellen SQL-Sitzung aktivieren.

Aktion und SQL Das Schema wird bei Bedarf angelegt. Der Suchpfad ermöglicht anschließend die Verwendung der Tabellennamen ohne Schemapräfix.

```
-- Teil A Schema erstellen
CREATE SCHEMA IF NOT EXISTS brewflow_oltp;
SET search_path TO brewflow_oltp;
```

Kontrolle Die Tabellenabfragen in Teil D greifen erfolgreich auf die Tabellen im Schema brewflow_oltp zu. In jedem neuen Query-Tab muss der Suchpfad erneut gesetzt werden; alternativ kann das Schemapräfix verwendet werden.

Ergebnis Das Schema brewflow_oltp ist erstellt und für die Projektabfragen ausgewählt.

Teil B Tabellen erstellen

Aufgabenstellung Die fünf Entitäten als relationale Tabellen mit Primärschlüsseln und den vorgesehenen Fremdschlüsselbeziehungen abbilden.

Schritt Zuerst customers, stores und products erstellen, danach orders und order_items. Diese Reihenfolge berücksichtigt die Abhängigkeiten.

Aktion und SQL Primärschlüssel identifizieren jeden Datensatz. Fremdschlüssel verbinden die Bestellungen mit Kunden und Filialen sowie die Positionen mit Bestellungen und Produkten. Eine Zeile in order_items entspricht einer Bestellposition.

```
-- Teil B Tabelle customers erstellen
CREATE TABLE customers (
    customer_id  INT PRIMARY KEY,
    first_name   VARCHAR(50)  NOT NULL,
    last_name    VARCHAR(50)  NOT NULL,
    email        VARCHAR(100),
    city         VARCHAR(50),
    loyalty_tier VARCHAR(20),   -- Bronze, Silver, Gold
    joined_date  DATE
);


-- Teil B Tabelle stores erstellen
CREATE TABLE stores (
    store_id     INT PRIMARY KEY,
    name         VARCHAR(100),
    city         VARCHAR(50),
    country      VARCHAR(50),
    opened_date  DATE
);


-- Teil B Tabelle products erstellen
CREATE TABLE products (
    product_id   INT PRIMARY KEY,
    name         VARCHAR(100),
    category     VARCHAR(50),   -- Coffee, Tea, Food, Merchandise
    unit_price   DECIMAL(6,2)
);
```

Teil B Tabellen und Beziehungen

```
-- Teil B Tabelle orders erstellen
CREATE TABLE orders (
    order_id     INT PRIMARY KEY,
    customer_id  INT REFERENCES customers(customer_id),
    store_id     INT REFERENCES stores(store_id),
    order_date   DATE,
    channel      VARCHAR(20),   -- in-store, mobile-app
    total_amount DECIMAL(8,2)
);


-- Teil B Tabelle order_items erstellen
CREATE TABLE order_items (
    order_item_id INT PRIMARY KEY,
    order_id      INT REFERENCES orders(order_id),
    product_id    INT REFERENCES products(product_id),
    quantity      INT,
    unit_price    DECIMAL(6,2)
);
```

Kontrolle Die fünf Tabellen werden in Teil D einzeln abgefragt. Die Bestellsummenprüfung verwendet die Beziehung orders.order_id zu order_items.order_id.

Tabelle	Primärschlüssel	Fremdschlüssel
customers	customer_id	—
stores	store_id	—
products	product_id	—
orders	order_id	customer_id, store_id
order_items	order_item_id	order_id, product_id

Ergebnis Im Schema brewflow_oltp sind customers, stores, products, orders und order_items angelegt.

Teil C Beispieldaten laden

Aufgabenstellung Die Tabellen mit den vorgesehenen Beispieldaten befüllen.

Schritt Die Stammdaten vor den Bestellungen und die Bestellungen vor ihren Positionen laden.

Aktion und SQL Die folgenden INSERT-Anweisungen enthalten die vollständigen Beispieldaten. Ihre Werte folgen der Spaltenreihenfolge der CREATE-TABLE-Anweisungen in Teil B. In Zeichenketten wird ein Apostroph durch zwei Apostrophe maskiert, beispielsweise bei O'Brien.

Die Anweisungen dokumentieren die bereits durchgeführte Erstbefüllung. Eine erneute Ausführung im befüllten Schema führt zu Konflikten mit vorhandenen Primärschlüsseln.

```
-- Teil C Beispieldaten für customers laden
INSERT INTO customers VALUES
(1,  'Sofia',  'Morales',   'sofia.m@example.com',  'Madrid',    'Bronze', '2022-03-15'),
(2,  'Lucas',  'Becker',    'lucas.b@example.com',  'Berlin',    'Silver', '2021-07-20'),
(3,  'Emma',   'Johansson', 'emma.j@example.com',   'Stockholm', 'Gold',   '2020-11-01'),
(4,  'Noah',   'Kim',       'noah.k@example.com',   'Seoul',     'Bronze', '2023-01-10'),
(5,  'Amelia', 'Patel',     'amelia.p@example.com', 'London',    'Silver', '2022-09-05'),
(6,  'Liam',   'Zhang',     'liam.z@example.com',   'Shanghai',  'Bronze', '2023-04-18'),
(7,  'Isla',   'Rossi',     'isla.r@example.com',   'Rome',      'Gold',   '2021-02-28'),
(8,  'Mateo',  'Oliveira',  'mateo.o@example.com',  'Lisbon',    'Silver', '2022-06-12'),
(9,  'Zara',   'Hassan',    'zara.h@example.com',   'Dubai',     'Bronze', '2023-08-22'),
(10, 'James',  'O''Brien',  'james.o@example.com',  'Dublin',    'Gold',   '2020-05-30');


-- Teil C Beispieldaten für stores laden
INSERT INTO stores VALUES
(1, 'BrewFlow Central',   'Madrid',    'Spain',    '2019-03-01'),
(2, 'BrewFlow Mitte',     'Berlin',    'Germany',  '2020-06-15'),
(3, 'BrewFlow Old Town',  'Stockholm', 'Sweden',   '2020-09-10'),
(4, 'BrewFlow Westside',  'London',    'UK',       '2021-04-22'),
(5, 'BrewFlow Riverside', 'Lisbon',    'Portugal', '2021-11-08');


-- Teil C Beispieldaten für products laden
INSERT INTO products VALUES
(1, 'Espresso',         'Coffee',      2.50),
(2, 'Cappuccino',       'Coffee',      3.80),
(3, 'Cold Brew',        'Coffee',      4.20),
(4, 'Matcha Latte',     'Tea',         4.00),
(5, 'Chamomile Tea',    'Tea',         2.80),
(6, 'Avocado Toast',    'Food',        6.50),
(7, 'Blueberry Muffin', 'Food',        3.20),
(8, 'BrewFlow Tumbler', 'Merchandise', 18.00);
```

Teil C Bestellungen laden

```
-- Teil C Beispieldaten für orders laden
INSERT INTO orders VALUES
(101, 1,  1, '2024-01-05', 'in-store',   7.00),
(102, 2,  2, '2024-01-07', 'in-store',   5.00),
(103, 3,  3, '2024-01-09', 'mobile-app', 10.70),
(104, 4,  1, '2024-01-12', 'in-store',   6.50),
(105, 5,  4, '2024-01-14', 'mobile-app', 4.00),
(106, 1,  1, '2024-01-20', 'in-store',   5.70),
(107, 6,  2, '2024-01-22', 'mobile-app', 4.20),
(108, 7,  5, '2024-01-25', 'in-store',   9.30),
(109, 8,  5, '2024-01-28', 'in-store',   2.50),
(110, 3,  3, '2024-02-02', 'mobile-app', 18.00),
(111, 9,  1, '2024-02-05', 'in-store',   3.80),
(112, 10, 4, '2024-02-08', 'mobile-app', 6.50),
(113, 2,  2, '2024-02-11', 'in-store',   6.60),
(114, 5,  4, '2024-02-14', 'mobile-app', 8.00),
(115, 1,  1, '2024-02-18', 'in-store',   4.20),
(116, 8,  5, '2024-02-22', 'in-store',   6.40),
(117, 4,  1, '2024-02-25', 'mobile-app', 7.80),
(118, 7,  5, '2024-02-28', 'in-store',   4.00),
(119, 10, 4, '2024-03-03', 'in-store',   22.00),
(120, 6,  2, '2024-03-07', 'mobile-app', 7.00),
(121, 3,  3, '2024-03-10', 'mobile-app', 9.00),
(122, 5,  4, '2024-03-14', 'in-store',   10.80);
```

Teil C Bestellpositionen laden

```
-- Teil C Beispieldaten für order_items laden
INSERT INTO order_items VALUES
-- Order 101: Cappuccino + Muffin = 7.00
(1001, 101, 2, 1, 3.80),
(1002, 101, 7, 1, 3.20),
-- Order 102: Espresso ×2 = 5.00
(1003, 102, 1, 2, 2.50),
-- Order 103: Cold Brew + Toast = 10.70
(1004, 103, 3, 1, 4.20),
(1005, 103, 6, 1, 6.50),
-- Order 104: Toast = 6.50
(1006, 104, 6, 1, 6.50),
-- Order 105: Matcha = 4.00
(1007, 105, 4, 1, 4.00),
-- Order 106: Espresso + Muffin = 5.70
(1008, 106, 1, 1, 2.50),
(1009, 106, 7, 1, 3.20),
-- Order 107: Cold Brew = 4.20
(1010, 107, 3, 1, 4.20),
-- Order 108: Chamomile + Toast = 9.30
(1011, 108, 5, 1, 2.80),
(1012, 108, 6, 1, 6.50),
-- Order 109: Espresso = 2.50
(1013, 109, 1, 1, 2.50),
-- Order 110: Tumbler = 18.00
(1014, 110, 8, 1, 18.00),
-- Order 111: Cappuccino = 3.80
(1015, 111, 2, 1, 3.80),
-- Order 112: Toast = 6.50
(1016, 112, 6, 1, 6.50),
-- Order 113: Cappuccino + Chamomile = 6.60
(1017, 113, 2, 1, 3.80),
(1018, 113, 5, 1, 2.80),
-- Order 114: Matcha ×2 = 8.00
(1019, 114, 4, 2, 4.00),
-- Order 115: Cold Brew = 4.20
(1020, 115, 3, 1, 4.20),
-- Order 116: Muffin ×2 = 6.40
(1021, 116, 7, 2, 3.20),
-- Order 117: Cappuccino + Matcha = 7.80
(1022, 117, 2, 1, 3.80),
(1023, 117, 4, 1, 4.00),
-- Order 118: Matcha = 4.00
(1024, 118, 4, 1, 4.00),
-- Order 119: Tumbler + Matcha = 22.00
(1025, 119, 8, 1, 18.00),
(1026, 119, 4, 1, 4.00),
-- Order 120: Cappuccino + Muffin = 7.00
(1027, 120, 2, 1, 3.80),
(1028, 120, 7, 1, 3.20),
-- Order 121: Espresso + Toast = 9.00
(1029, 121, 1, 1, 2.50),
(1030, 121, 6, 1, 6.50),
-- Order 122: Cappuccino ×2 + Muffin = 10.80
(1031, 122, 2, 2, 3.80),
(1032, 122, 7, 1, 3.20);
```

Kontrolle Die COUNT-Abfragen in Teil D vergleichen die tatsächlich geladenen Datensätze mit den Vorgaben. Die zweite Prüfung gleicht die Bestellbeträge mit den Positionen ab.

Ergebnis Geladen wurden 10 Kunden, 5 Filialen, 8 Produkte, 22 Bestellungen und 32 Bestellpositionen.

Teil D Validierung

Prüfung 1 Zeilenanzahlen

Aufgabenstellung Die Vollständigkeit der Beispieldaten anhand der vorgegebenen Zeilenanzahlen prüfen.

Schritt Die Anzahl der Datensätze je Tabelle ermitteln und mit den Sollwerten vergleichen.

Aktion und SQL Der Suchpfad wird vor der Prüfung explizit gesetzt.

```
-- Teil D Prüfung 1 Zeilenanzahlen prüfen
-- Erwartet customers 10 stores 5 products 8
-- Erwartet orders 22 order_items 32
SET search_path TO brewflow_oltp;

SELECT 'customers' AS tbl, COUNT(*) AS rows FROM customers
UNION ALL
SELECT 'stores', COUNT(*) FROM stores
UNION ALL
SELECT 'products', COUNT(*) FROM products
UNION ALL
SELECT 'orders', COUNT(*) FROM orders
UNION ALL
SELECT 'order_items', COUNT(*) FROM order_items;
```

Kontrolle Die Abfrage liefert fünf Ergebniszeilen. Soll- und Istwert sind in jeder Zeile identisch.

Tabelle	Soll	Ist	Ergebnis
customers	10	10	Bestanden
stores	5	5	Bestanden
products	8	8	Bestanden
orders	22	22	Bestanden
order_items	32	32	Bestanden

Ergebnis Prüfung 1 ist bestanden. Alle fünf Tabellen enthalten die erwartete Anzahl von Datensätzen.

Teil D Nachweis der Zeilenanzahlen

Abbildung 1 zeigt die ausgeführte COUNT-Abfrage und die bestätigten Ergebnisse im SQL-Client.

```
250 SET search_path TO brewflow_oltp;
251
252 SELECT 'customers' AS tbl, COUNT(*) AS rows
253 FROM customers
254
255 UNION ALL
256
257 SELECT 'stores', COUNT(*)
258 FROM stores
259
260 UNION ALL
261
262 SELECT 'products', COUNT(*)
263 FROM products
264
265 UNION ALL
266
267 SELECT 'orders', COUNT(*)
268 FROM orders
269
270 UNION ALL
271
272 SELECT 'order_items', COUNT(*)
```

Auto Commit | Manual

	tbl	rows
1	customers	10
2	stores	5
3	products	8
4	orders	22
5	order_items	32

Teil D Bestellsummen validieren

Aufgabenstellung Sicherstellen, dass jede gespeicherte Bestellsumme der Summe ihrer Bestellpositionen entspricht.

Schritt orders und order_items über order_id verbinden. Für jede Bestellung quantity × unit_price summieren und mit total_amount vergleichen.

Aktion und SQL Die Abfrage gibt pro Bestellung den gespeicherten Betrag, den berechneten Positionsbetrag und das Vergleichsergebnis match zurück.

```
-- Teil D Prüfung 2 Bestellsummen abgleichen
-- Erwartet 22 Zeilen und überall match = true
SET search_path TO brewflow_oltp;

SELECT
    o.order_id,
    o.total_amount AS order_total,
    SUM(oi.quantity * oi.unit_price) AS items_total,
    o.total_amount = SUM(oi.quantity * oi.unit_price) AS match
FROM orders o
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY
    o.order_id,
    o.total_amount
ORDER BY
    o.order_id;
```

Kontrolle Das Ergebnis enthält alle 22 Bestellungen mit den IDs 101 bis 122. Bei jeder Bestellung sind order_total und items_total gleich; match ist durchgehend true.

Kontrollmerkmal	Bestätigtes Ergebnis
Geprüfte Bestellungen	22 von 22
Übereinstimmende Bestellsummen	22
Abweichende Bestellsummen	0
Vergleich match	true in allen 22 Zeilen

Ergebnis Prüfung 2 ist bestanden. Alle 22 Bestellsummen stimmen mit den zugehörigen Bestellpositionen überein.

Damit sind die Einrichtung, die Befüllung und die beiden vorgesehenen Validierungen für BrewFlow Final Project Teil 1 erfolgreich abgeschlossen.

Teil D Nachweis der Bestellsummen

Abbildung 2 zeigt alle 22 Ergebniszeilen. Die Spalte match enthält für jede Bestellung den Wert true.

	order_id	order_total	items_total	match
1	101	7.00	7.00	true
2	102	5.00	5.00	true
3	103	10.70	10.70	true
4	104	6.50	6.50	true
5	105	4.00	4.00	true
6	106	5.70	5.70	true
7	107	4.20	4.20	true
8	108	9.30	9.30	true
9	109	2.50	2.50	true
10	110	18.00	18.00	true
11	111	3.80	3.80	true
12	112	6.50	6.50	true
13	113	6.60	6.60	true
14	114	8.00	8.00	true
15	115	4.20	4.20	true
16	116	6.40	6.40	true
17	117	7.80	7.80	true
18	118	4.00	4.00	true
19	119	22.00	22.00	true
20	120	7.00	7.00	true
21	121	9.00	9.00	true
22	122	10.80	10.80	true